

Active/Standby High Availability for the KubeSlice Controller

Cross-cluster leader election, state mirroring and automatic failover for the KubeSlice management plane

Mentee	Sumanth D (github.com/sumanthd032)
Mentors	Gourish Biradar (gourishkb), Prabhu Navali (pnavali), Rahul Kumar (Rahul-D78)
Project	kubeslice-controller #305 — LFX Mentorship: HA Active/Standby
Term	24 June – 31 August 2026
Organisation	KubeSlice (Avesha)
Report date	5 September 2026

Status at submission. The feature is complete and merged. Twenty-three pull requests across four upstream repositories were merged into the project's `ha-integration-branch` on 29 August 2026; four final integration-to-master pull requests are open, awaiting the mentors' merge. The implementation was validated on a three-cluster Kind topology and on two real managed clusters in different clouds (Linode LKE and Oracle OKE), including one *unplanned* failover triggered by Oracle reclaiming the Active hub's nodes overnight.

Contents

1	Executive summary	1
2	Problem, scope and requirements	1
3	Architecture	3
4	Implementation	4
5	Operational surface	7
6	Verification	8
7	Defects found and fixed	10
8	Known limitations and future work	12
9	Delivery summary	12
10	Conclusion	13
A	Running the tests and deploying a hub pair	14

1 Executive summary

The KubeSlice management plane runs on a single Kubernetes cluster, which makes that cluster a single point of failure. If it is lost, every registered worker keeps its existing data plane but loses all configuration management — no new slices, no reconciliation, no VPN key rotation, no cluster registration — until the hub is rebuilt by hand and every worker re-pointed.

This project makes the KubeSlice Controller survive the loss of an entire hub cluster. Two hubs run the controller at once. One is **Active**: it holds a Kubernetes **Lease** on its own API server and is the only hub permitted to write. The other is **Standby**: every reconciler refuses to write, and instead of reconciling it continuously mirrors the Active’s configuration into its own API server over a remote client. When the Active stops renewing its Lease, the Standby detects the staleness, runs a guarded promotion sequence, takes leadership, and publishes its own endpoint into each registered cluster’s status. Worker operators read that field, notice the change, and reconnect to the promoted hub with no operator action.

Delivered: cross-cluster leader election with a per-call write fence on all nine mutating reconcilers; a workqueue-backed Active→Standby mirror of ten resource kinds plus worker credentials, with bidirectional anti-entropy; guarded automatic failover with promotion persistence across restarts; worker-side failover following with an explicit trust and anti-flap ruleset; 18 metrics, 6 events, an ADR, an operator runbook and a test-suite reference; and four layers of verification. Deployments that do not opt in are byte-for-byte unaffected.

Measurement	Result	Environment
Failover detection + promotion	21 s (budget 20 s)	Kind, 15 s lease + 5 s padding
Failover, cross-cloud LKE→OKE	25 s (budget 30 s)	Real managed clusters
Promotion sequence itself	0.106 s	Kind, instrumented
Mirror propagation	<1 s / 2.5–3.0 s	Kind / cross-cloud (WAN-bound)
Worker reconnect after promotion	~50 s	Real clusters, both workers
Unplanned failover (node reclaim)	21 s, unattended	Oracle OKE → Linode LKE

Table 1: Measured failover characteristics. Every figure was observed live, not modelled.

2 Problem, scope and requirements

KubeSlice is hub-and-spoke. A **controller** (the hub) owns the user-facing custom resources — **Project**, **Cluster**, **SliceConfig**, **ServiceExportConfig**, **SliceQoSConfig**, **VpnKeyRotation** — and decomposes them

into per-worker resources (`WorkerSliceConfig`, `WorkerSliceGateway`, `WorkerServiceImport`). A **worker operator** on every participating cluster connects outward to one hub endpoint, watches the resources addressed to it, and programs the local data plane.

2.1 Multi-cluster, not multi-node

This is the most important framing point, because the codebase contains a trap. `main.go` already had a `--leader-elect` flag: controller-runtime’s standard *in-cluster*, *pod-level* election, which decides which *replica pod* is active within one cluster. It is scoped entirely inside a single API server and does nothing for the loss of that API server. This project implements election between two *separate* Kubernetes clusters, each with its own API server, etcd, and in general its own cloud and region. The existing flag is untouched and orthogonal.

	Existing <code>--leader-elect</code>	This project
Competitors	Pods in one Deployment	Controllers in two clusters
Lease lives on	The one shared API server	The Active cluster’s own API server
Survives	Pod crash, node loss	Loss of a cluster, region or cloud
Visible to workers	No (endpoint unchanged)	Yes (endpoint changes; workers follow)
State to replicate	None (shared etcd)	All hub configuration, cross-cluster

Table 2: Why the pre-existing leader-election flag does not address this problem.

2.2 Requirements

R1 Single writer — exactly one hub may write at any time, re-checked continuously rather than granted once at start-up. **R2 State replication** — the Standby mirrors the Active’s configuration so a promoted hub starts from current state, not an empty cluster. **R3 Heartbeating** — the Standby observes the Active’s liveness across the cluster boundary. **R4 Failover** — on detected failure the Standby takes leadership without operator action. **R5 Worker re-pointing** — workers discover and connect to the promoted hub. **R6 Backward compatibility** — a deployment that never opts in observes no behavioural change at all. **R7 Observability** — operators can tell which hub is Active, whether the mirror is healthy, and why a promotion did or did not happen.

R6 shaped much of the implementation: the mode flag defaults to `standalone`, in which the elector reports leadership unconditionally, the mirror never starts, and the `status.activeController` publisher is deliberately *not* started — if it were, every existing non-HA deployment would begin writing a new status field. A nil elector is treated as standalone.

2.3 Non-goals

- **Split-brain resolution.** A partition that isolates the two hubs while both stay healthy lets the Active keep renewing while the Standby — which can no longer see it — promotes. Resolving this needs an arbiter or a quorum. Section 8 reports what happened in practice.
- **More than two hubs**, and **automatic demotion** of a recovered Active: a hub reads only its own Lease, and the two Leases are separate objects on separate API servers, so there is no shared record of who won. Demotion is an operator action.
- **Data-plane failover.** Slice gateways and tunnels are worker-local and already survive hub loss. A static test asserts the failover package never imports gateway code.

3 Architecture

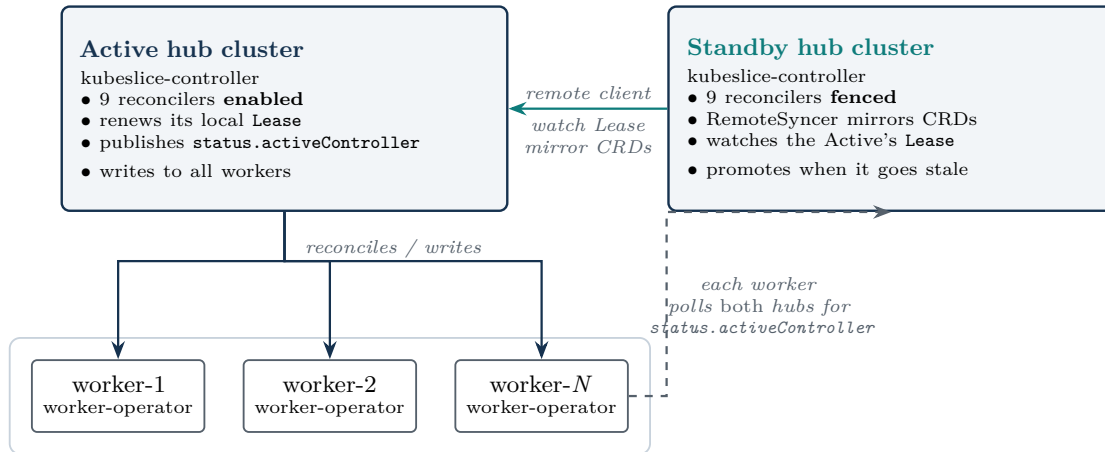


Figure 1: Steady state. Only the Active writes. The Standby holds one remote client to the Active’s API server, over which it both watches the Lease and runs the mirror informers. Workers read the promoted endpoint from their own **Cluster** CR status, which *both* hubs carry because the mirror replicates it.

The moving parts are: a **ClusterLeaderElector** on both hubs (owns the mode, renews the local Lease on the Active, watches the remote Lease on the Standby, answers `IsLeader()` from an atomic flag kept fresh by a background loop); the reconciler write fence; a **RemoteSyncer** and a bidirectional prune pass on the Standby; the promotion state machine; an **ActiveControllerPublisher** on whichever hub leads; a **ReconcileKicker** that wakes a promoted hub’s reconcilers; **ResumeAsActive** at start-up; and, in the worker operator, a hub **Resolver** and a **Follower**.

3.1 Design decisions

Recorded in `docs/adr/ha-active-standby-adr.md` (362 lines), merged as [#405](#) and amended three times as implementation refined the design.

Table 3: The eight ADR decisions.

#	Question	Decision
1	Where does the Lease live?	On the Active cluster’s own API server, in the controller’s own namespace. Not an external store: a hub that cannot reach its own API server cannot renew, and therefore cannot write — fencing falls out of the failure itself.
2	How does the Standby get credentials?	A kubeconfig stored as a Secret in the Standby and mounted into the pod (<code>--ha-active-kubeconfig</code>). A least-privilege sample ClusterRole ships at <code>config/ha/</code> .
3	How is election implemented?	A purpose-built ClusterLeaderElector . The library primitive assumes all candidates share one API server; here they do not, and the Standby must <i>observe</i> a Lease it can never acquire.
4	How is only-Active-writes guaranteed?	Per-call gating. Every mutating Reconcile() begins <code>if !IsLeader() { return }</code> , evaluated on every invocation. A nil elector means standalone.
5	What triggers promotion?	The remote Lease’s <code>renewTime</code> failing to advance beyond <code>leaseDuration + padding</code> , measured from a <i>cached</i> last-good read so an unreachable API server ages exactly like a dead pod. Three guards must then pass.
6	What is mirrored, and how?	Ten hub-side kinds plus four credential kinds (Table 4). Informers on the Active feed a rate-limited workqueue; each item is re-read at dequeue and written locally with <code>resourceVersion</code> , <code>uid</code> and owner references stripped as appropriate.
7	How do workers find the new Active?	Pull, not push. The leading hub writes <code>status.activeController</code> into each Cluster CR on its own API server; the mirror carries it to the other hub; the worker — which already watches its own Cluster CR — reads it from whichever hub answers.

Table 3: (continued)

#	Question	Decision
8	Split-brain	Explicit non-goal, documented rather than mitigated (Section 8).

Decision 7 contradicted the issue text, and the ADR won. Issue #297 step 7 specified pushing `cluster.Spec.ControllerEndpoint` to every worker over per-worker remote clients, and named a `ClusterController` CR. Neither the field nor the CR exists in either repository. A push model would also require the promoting hub to hold credentials to every worker and succeed against all of them *during* promotion. The pull model reuses a watch the worker already runs.

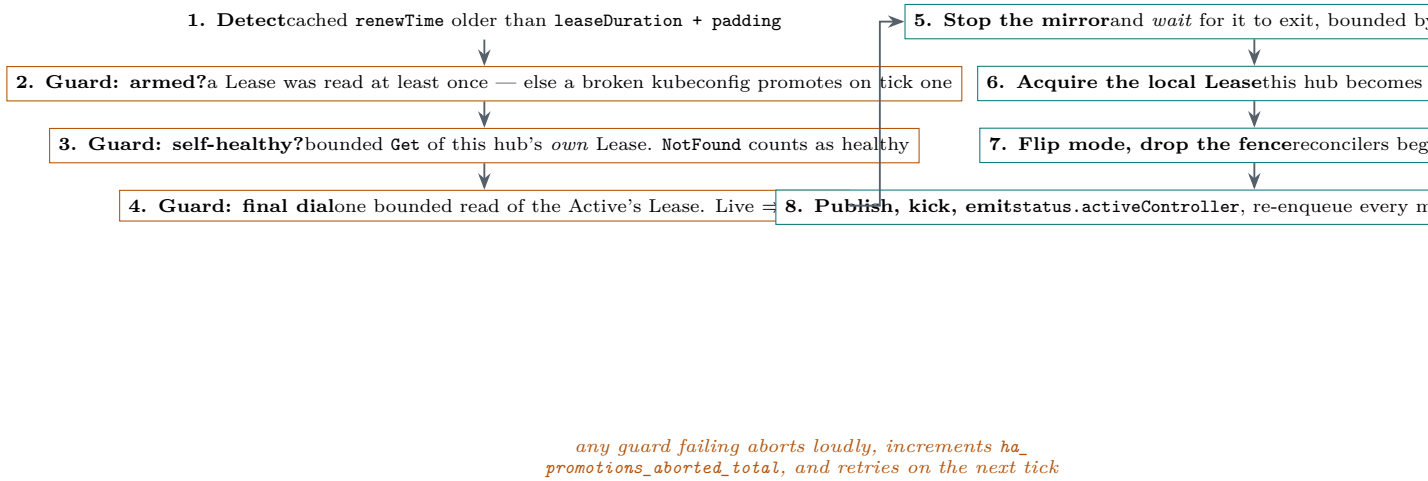


Figure 2: The promotion sequence. Step ordering is load-bearing: the mirror is stopped and *confirmed exited* before the Lease is acquired, because a mirror still running against a live-but-pod-dead Active would overwrite the newly promoted hub’s own writes.

4 Implementation

4.1 #294 — Leader election and write fencing

`pkg/ha` was created here. `ClusterLeaderElector` is built from an `Options` struct rather than the three positional arguments the issue sketched, because every timing must be configurable for testing and for widely varying inter-cluster latency. `IsLeader()` reads an `atomic.Bool`, so the hot path in nine reconcilers costs an atomic load rather than an API call. Exactly one background loop runs per mode: `StartLeaseRenewal` on the Active, which *drops* leadership if renewal has not succeeded within `--ha-renew-deadline` — a hub that lost its API server stops believing it leads, without being told — and `WatchRemoteLease` on the Standby.

The fence itself is three lines at the top of nine mutating reconcilers (`Project`, `Cluster`, `SliceConfig`, `ServiceExportConfig`, `SliceQoSConfig`, `VpnKeyRotation`, `WorkerSliceConfig`, `WorkerSliceGateway`, `WorkerServiceImport`):

```

if r.HAElector != nil && !r.HAElector.IsLeader() {
    logger.Debug("standby mode, skipping reconcile")
    return ctrl.Result{}, nil
}

```

A dedicated test attaches a zap observer and asserts the skip is logged exactly once per `Reconcile` call, which proves the gate is per-invocation — the substance of R1.

4.2 #295 — Active-to-Standby state mirror

The initial design was a `cache.Cache` with fire-and-forget event handlers. Two properties killed it: the issue’s own acceptance criterion required the syncer to retry without crashing, and a fire-and-forget handler has nowhere to retry from; and every mirrored type declares `+kubebuilder:subresource:status`, so a plain `Update()` silently drops `.status`.

The shipped design puts a rate-limited `workqueue` — the same primitive controller-runtime’s own `Controller` uses internally — between the informer callbacks and the mirror logic, and issues an explicit

`Status().Update()` after every write. Retry then comes for free, which let `Namespace` become an ordinary row in the resource table instead of a special synchronous first pass: ordering self-heals through retry. A second `manager.Manager` with a typed reconciler per kind was considered and rejected — more idiomatic-looking, no correctness gain, considerably more ceremony.

Kind	Group	Treatment
Namespace	core	Label-scoped to project namespaces; status <i>not</i> mirrored
Project, Cluster, SliceConfig, ServiceExportConfig, SliceQoSConfig, VpnKeyRotation	controller.kubeslice.io	Cluster carries <code>status.activeController</code> ; <code>VpnKeyRotation</code> has owner references stripped
WorkerSliceConfig, WorkerSliceGateway, WorkerServiceImport	worker.kubeslice.io	Decomposed per-worker resources
Secret	core	Field-selected; SA-token Secrets mirrored as <i>shells</i> only
ServiceAccount, Role, RoleBinding	core, rbac...k8s.io	Label-scoped, owner references stripped

Table 4: The mirrored resource set: configuration above, and below it the credential material a promoted hub needs so that workers keep authenticating.

The issue’s own resource list was wrong, and the code says so. Issue [#295](#) names `Slice`, `SliceGateway` and `ServiceExport`. None exists in `kubeslice-controller` — they are worker-cluster data-plane CRDs in group `networking.kubeslice.io`, owned by the separate worker-operator repository and irrelevant to hub-to-hub mirroring. The real set was derived from `apis/controller/v1alpha1` and `apis/worker/v1alpha1`, and `mirror_set.go` carries a comment explaining the divergence so the next reader does not “fix” it back.

Drift backstop. Informers miss events, objects get deleted directly on the Standby, and keys sit deep in rate-limiter backoff. A pass every `--ha-sync-interval` diff’s both directions: labelled mirrors whose original is gone are enqueued for deletion *through the same workqueue*, so the worker re-reads the Active at dequeue and the conflict guard still applies; and Active objects with no local mirror are enqueued for creation, bypassing backoff. This was measured to matter — a fixed failure otherwise took minutes to heal because the next retry was 164s away. The pass waits for the remote cache to sync first: an unsynced cache lists empty, which would read as “everything was deleted” and prune every mirror. A failed list skips that kind for the round and increments `ha_sync_errors_total` — never delete on partial information.

Credentials and the shell trick. A `ServiceAccount` token is signed by the *issuing* cluster’s key, so an Active-minted token presented to the Standby is simply `Unauthorized` — verified as a negative control on a live cluster. The mirror therefore copies the token Secret’s *shell* (name, namespace, type, and the `service-account.name` annotation), dropping `.data` and the `service-account.uid` annotation, and lets the Standby’s own token controller mint a valid local token into it. Two further properties were needed, both found by reading the mirror engine rather than by testing: the row must be `CreateOnly`, because the mirror does an unconditional full `Update` and the informer resyncs every ten minutes, so a payload with no `.data` would wipe the minted token on a timer; and the `service-account.uid` annotation must be dropped, because the token controller validates it against the *local* `ServiceAccount`’s UID and deletes the Secret on mismatch — whereupon the prune reverse-diff restores it, producing a hot loop.

4.3 [#297](#) — Failover and promotion

Detection. The first design had three fields and a five-row verdict table branching on whether the Active’s pod or its API server had died. It collapsed to a single cached `*coordinationv1.Lease` plus an explicit nil check, on the observation that the two cases are the *same event*: in both, the newest `renewTime` the Standby holds stops advancing. A retained stale cache ages on its own, the already-shipped `isLeaseStale` covers both paths, and arming falls out for free as the nil case.

A footgun worth stating explicitly: `isLeaseStale(nil, ...)` returns *true*. The nil check must therefore be a separate statement *before* it — otherwise a typo in the `kubeconfig` path means an unreachable Active on tick one, and instant false promotion.

Guards. Three failure modes look byte-identical from the Standby: the Active’s API server is dead; the hubs

are partitioned; the Standby’s *own* network is dead. *Armed* requires a Lease to have been read at least once, separating “the Active died” from “this Standby was never configured”. *Self-health* is a bounded `Get` of the Standby’s own Lease through its local client — no new RBAC, and `NotFound` counts as healthy because the server answered and no Lease exists before a first promotion. This is the only cheap discriminator for the third mode. The *final dial* is a polling-race guard, *not* a split-brain guard: in a genuine partition the dial travels the same broken path and fails, so the Standby promotes anyway. The ADR previously described it as a partition guard; that was corrected.

Sequencing. Both the issue text and the ADR originally put “stop the mirror” *after* the mode flip. That is a real dual-writer bug when the Active’s pod is dead but its API server is alive: every mirrored object carries the sync label, so the conflict guard permits overwrite and the prune reverse-diff resurrects deletions — the mirror would fight the promoted hub’s own reconcilers. Every step that waits on another component is bounded by `--ha-promotion-grace-period`, for a reason testing found rather than reasoning (defect D11).

Publishing, and reconciling afterwards. `ActiveControllerPublisher` writes `status.activeController` into every `Cluster` CR on a 30-second loop and once inline during promotion. It reuses the existing `--controller-end-point` flag rather than adding one, refuses to publish the shipped placeholder value, and re-reads after writing to confirm the field persisted — a check added after a real cluster silently pruned it for four hours (D14). Its convergence comparison excludes `lastUpdated`, or every tick would write to every `Cluster` CR forever. Separately, flipping `IsLeader()` wakes nothing on its own: the fence returns without requeueing and the default informer resync is ten hours, so a `ReconcileKicker` pushes every mirrored object into a per-GVK `source.Channel` consumed through `WatchesRawSource`. One shared channel would have been wrong — each reconciler must receive its own kind.

4.4 #298, #299 — Observability, runbook and end-to-end tests

Eighteen metrics, six events, and `docs/ha-runbook.md` (547 lines) covering verification, failover simulation, credential rotation, sync-lag and stuck-promotion diagnosis, and split-brain recovery, with Prometheus alerting expressions. Two decisions are worth recording: role-specific gauges are left *unset* on the role they do not apply to rather than zeroed, because a 0 on a timestamp gauge reads as 1970 and breaks every `time()` - `metric` expression (the two exceptions, `ha_leader_status` and `ha_armed`, are initialised precisely *because* 0 is their alertable value); and aborts route through a single `abortPromotion(ctx, reason)` that increments the counter *and* emits the event, so the two cannot drift.

#299 adds a Go end-to-end suite under `test/e2e` behind an `e2e` build tag and a `make test-e2e-ha` target. It creates two disposable Kind clusters, deploys the controller to both in Active and Standby modes, and runs four scenarios: baseline mirror sync; a transient RBAC blip that must *not* promote; a real failover; and reconciliation resuming on the promoted hub. `docs/ha-testing.md` (774 lines) documents every test with a coverage map back to #293–#299.

4.5 Worker operator — #467, #468, #469

The worker polls both candidate hubs and applies an ordered ruleset: an unreachable hub gets no say; a reachable hub with no `status.activeController` gets no say (the ordinary non-HA path, logged at info, not warned); **a claim naming an endpoint outside the configured candidate set is rejected** — the trust boundary, so a compromised or misconfigured hub cannot redirect a worker to an arbitrary address; agreement wins, and agreement is the *normal* case because the Standby mirrors the Active’s `Cluster` CR; disagreement is resolved by the fresher `lastUpdated`; and if nothing is usable, **change nothing** — never disconnect on ambiguity. Anti-flap requires *N* consecutive agreeing polls (default 2), deliberately excluding `lastUpdated` from the comparison, since the Active republishes on a timer and including it would reset the counter every poll so no switch could ever confirm.

The load-bearing finding on the worker side: the endpoint and the credentials were bound at *different times*. `HUB_HOST_ENDPOINT` is read inside the client constructors, but the token and CA file paths are package variables evaluated before `main` runs. Setting the environment variable from `main` would have moved the address but not the token — a client aimed at hub B authenticating as hub A. The fix threads an explicit `hub.Connection{Endpoint, TokenFile, CAFile}` value through both hub connections.

The switch itself is a graceful process exit, letting the kubelet restart the pod against the new hub. That is established behaviour there — `manager.Start` already calls `os.Exit(1)` on every error path — and gateways and tunnels are separate pods, so the data plane is untouched. #469 adds `ControllerConnected` and `ControllerEndpointSynced` conditions, events, and two metrics; the first reconcile after a resolved switch reports `ReconnectedAfterFailover`, every one after that plain `Connected`. One limitation is architectural and is documented rather than papered over: `DialFailed` and `CertVerificationFailed` are real, unit-tested

reason constants but can never be *persisted* live, because writing “this hub is unreachable” needs the very connection that just failed.

4.6 Four repositories, not one

Repository	Why it had to change
kubeslice-controller	The hub implementation: <code>pkg/ha</code> , the fence, the publisher, docs, e2e.
worker-operator	The resolver, failover following, connection health.
apis	The worker operator does <i>not</i> use the controller’s <code>Cluster</code> type — it imports the shared <code>apis</code> module, so <code>ActiveController</code> and <code>Conditions</code> had to be added to <code>ClusterStatus</code> there too.
kubeslice (Helm charts)	The chart ships its own copy of the <code>Cluster</code> CRD, predating both new status fields. Kubernetes prunes unknown fields <i>silently</i> , so without this change every write of <code>status.activeController</code> is discarded on a stock install and the whole discovery mechanism is inert.

Table 5: The four repositories this feature touches.

CRD pruning was the most expensive recurring trap in this project. It appeared four separate times — the chart’s `Cluster` CRD, the controller’s own `ClusterStatus`, the `apis` repository’s shipped CRDs, and the worker CRDs for unrelated gateway fields. Every instance looks identical from outside: the write returns success, the controller logs success, and the field is simply absent. The eventual mitigation — the publisher’s read-after-write check — exists so the *next* person sees an error naming the likely cause instead of silence.

5 Operational surface

Thirteen new flags, all prefixed `ha-`. None is required: omitting all of them leaves the controller in `standalone` mode, behaving exactly as before this project. Detection latency is bounded by `lease-duration` + `padding-seconds` + `retry-period`, and total failover by that plus the promotion sequence, itself bounded by four grace periods.

Table 6: The thirteen HA flags.

Flag	Default	Purpose
<code>--ha-mode</code>	<code>standalone</code>	<code>active</code> , <code>standby</code> or <code>standalone</code> . An unrecognised value is fatal, not a fallback — see D9
<code>--ha-identity</code>	<code>hostname</code>	Stable per-hub identity in the Lease and in <code>status.activeController</code> . Under a Deployment the hostname is the pod name and changes on every restart, so pin it
<code>--ha-active-kubeconfig</code>	—	Path to the Active hub’s kubeconfig; required in standby mode
<code>--ha-lease-namespace</code>	<code>own ns</code>	From <code>KUBESLICE_CONTROLLER_MANAGER_NAMESPACE</code> via the downward API. Hard-coding this caused a self-fencing deadlock — see D1
<code>--ha-lease-duration</code>	<code>15 s</code>	HA Lease duration
<code>--ha-renew-deadline</code>	<code>10 s</code>	How long the Active may fail to renew before releasing leadership
<code>--ha-retry-period</code>	<code>2 s</code>	Interval between renew (Active) or watch (Standby) attempts
<code>--ha-padding-seconds</code>	<code>5 s</code>	Extra buffer before a Standby calls the Active’s Lease stale
<code>--ha-sync-workers</code>	<code>4</code>	Workers draining the mirror workqueue
<code>--ha-sync-interval</code>	<code>60 s</code>	Anti-entropy pass interval

Table 6: (continued)

Flag	Default	Purpose
-	5s	Bounds <i>every</i> networked Lease read: the periodic poll, the self-health check, the final dial. Unbounded, this stalls detection entirely — see D7
-ha-promotion-dial-timeout	10s	Bounds each promotion step that waits on another component. A sequencing budget, unrelated to
-		--ha-padding-seconds
-ha-promotion-grace-period	SA CA	This hub’s own API server CA, published in
-	path	status.activeController.caBundle. Unreadable is not fatal
-ha-self-ca-bundle-path		

Table 7: The eighteen metrics, all prefixed `kubeslice_controller_ha_`.

Metric	Type	Meaning
leader_status	gauge	1 if this instance holds HA leadership, 0 if not
armed	gauge	1 if this Standby has ever read the Active’s Lease and could promote
lease_last_renew_time_seconds	gauge	Last successful local Lease renewal (Active)
lease_renew_errors_total	counter	Failed local Lease renewals
remote_lease_age_seconds	gauge	Age of the newest Lease read from the Active
remote_lease_reads_total	counter	Remote Lease reads, by result
sync_lag_seconds	histogram	Active-side change to Standby-side appearance
sync_errors_total	counter	Mirror failures, by kind and operation
sync_queue_depth	gauge	Keys waiting in the mirror workqueue
prune_resurrected_total	counter	Active objects re-enqueued for a missing mirror
prune_last_run_timestamp_seconds	gauge	Last completed anti-entropy pass
failover_total	counter	Completed promotions
failover_detection_seconds	histogram	Last observed renewal to commitment to promote
promotion_duration_seconds	histogram	Promotion wall clock, by outcome
promotion_step_duration_seconds	histogram	Per-step promotion wall clock
promotions_aborted_total	counter	Promotions considered then refused, by reason
last_promotion_timestamp_seconds	gauge	When this process last promoted
active_publish_errors_total	counter	Failed <code>status.activeController</code> writes — <i>this one means “failover may work without any worker noticing”</i>

Six events are emitted, all attached to the HA Lease so the recorder derives the controller’s own namespace: `HABecameActive` and `HABecameStandby` at start-up, `HAPromotedToActive` on a completed promotion, `HAPromotionAborted` (Warning) with a reason, `HALeadershipLost` (Warning) when the Active misses its renew deadline, and `HAMirrorSyncFailed` (Warning) once per failure *episode* — gated on `NumRequeues == 0`, because the recorder aggregates repeats into a count but each call is still an API write, so per-attempt emission would be a write storm during any sustained outage.

A least-privilege sample `ClusterRole` for the Standby’s remote identity ships at `config/ha/`, deliberately *outside* the kustomization graph — both `config/rbac` and `config/default` enumerate their files explicitly — so it can never be accidentally applied to the Standby’s own cluster. Its README states an honest caveat rather than an aspirational one: RBAC cannot scope Secret access by `.type` or by namespace-name pattern, so a `ClusterRole` binding grants cluster-wide read of *all* Secrets on the Active. The per-namespace `RoleBinding` alternative is documented. An earlier draft understated this as “project namespaces only”; it was corrected.

6 Verification

Four layers were used, each catching a class the layer below structurally cannot: unit tests with fake clients under `-race` and `-shuffle`; a Go end-to-end suite over two disposable Kind clusters; a six-phase shell suite

over a real multi-cluster topology; and two real managed clusters in different clouds, each running a hub *and* a worker.

A fake client is not a small API server. Five defects in this project were invisible to unit tests *by construction*, not by oversight: a fake client does not enforce immutable-field validation, does not run garbage collection, does not run the token controller, does not prune unknown CRD fields, and does not model Prometheus collection. Every one of those produced a real bug. Any test double that accepts writes a real API server would reject gives false confidence exactly where correctness matters most.

Package	Files	Tests	Lines	Coverage
pkg/ha (controller)	31	185	8 570	88.4 %
controllers/* write fence	1	1	62	—
test/e2e (controller)	11	1 (+4 scenarios)	1 269	—
pkg/hub/resolver (worker)	4	25	1 178	96.7 %
pkg/hub/failover (worker)	4	15	800	85.9 %
Total	51	227	11 879	

Table 8: Test inventory, re-run for this report on 5 September 2026. Line counts include test code. All suites are `-race` clean; `pkg/ha` is additionally clean under `-shuffle=on` and `-count=2`, which matters because its metrics are package globals.

Run	Result	Coverage
Kind, 3 clusters (18 Aug)	9 phases green	Reset, worker wiring, baseline, failover, worker-follows, restore
Real cross-cloud (18 Aug)	92 assertions, 0 failures	Preflight 17, baseline 14, workers 12, failover 20, follow 18, restore 11 — then re-verified green in the <i>reverse</i> direction
Kind, pre-demo (3 Sep)	197 assertions, 0 failures	Two complete failover cycles, four minutes after a cold reboot

Table 9: Multi-cluster suite results. The suite asserts only on Leases, CR fields, events and logs — never on the metrics endpoint — so it exercises the surface an operator would inspect.

Two pre-existing failures in these repositories are *not* caused by this work and were deliberately not absorbed into it: the controller’s `TestAPIs` envtest suite requires control-plane binaries that are not installed, and `go vet ./service/...` fails on `master` with `undefined: util.Client`. Both fail identically on an untouched checkout.

6.1 Real multi-cloud validation, and one unplanned failover

Two mentor-provided managed clusters, both bare at handover: **Linode LKE** (`us-mia-2`, Kubernetes v1.35.6) as Active hub plus worker, and **Oracle OKE** (v1.34.2, preemptible nodes) as Standby hub plus worker. Each cluster running both roles made the test representative — two real workers exercise the dual-hub resolver and the mirrored token shells on both. Measured API-server round trip between them was 30–192 ms. Cross-cloud failover completed in **25 s** against a 30 s budget, with both workers writing to the promoted hub within roughly 50 s and the full sequence in the documented order, mirror confirmed stopped before the Lease was acquired.

Then, one day, Oracle reclaimed both preemptible nodes underneath the Active hub. Nothing was staged and nobody was watching.

```

20:22:18Z hub-oke last successful Lease renewal
... hub-oke nodes reclaimed; API server and controller gone
20:22:39Z hub-lke active hub lease is STALE
20:22:39Z hub-lke final dial reached the active hub but its lease is still stale; proceeding
20:22:39Z hub-lke mirror stopped -> lease acquired -> mode active -> activeController
published -> LeadershipAcquired -> PROMOTED -> watch stopping
... worker-lke, worker-oke follow to hub-lke

```

All seven steps completed within the same second, 21 s after the last renewal. The final dial is the detail worth noting: it *reached* the dying cluster’s API server, read the Lease, found it genuinely stale, and said so — confirming real failure rather than mistaking a WAN partition for a dead hub. This is the closest thing to production evidence the project has, and it was not arranged.

7 Defects found and fixed

Most of the engineering effort went into defects, not features. The catalogue is more informative than the feature list: it shows which assumptions were wrong and which layer caught each one. All were fixed before the code merged.

Table 10: Selected defects found and fixed during the project.

ID	Defect	Consequence	Found by
D1	Lease namespace hard-coded while the leader-election Role is namespaced	Self-fencing deadlock. Deployed into any non-default namespace, the Active had no <code>leases</code> RBAC there, could never renew, and fenced itself permanently	Code review
D2	Unscoped <code>Namespace</code> mirroring	The remote cache watched <i>every</i> namespace including <code>kube-system</code> ; an Active-side delete would cascade-delete it, and its contents, on the Standby	Code review
D3	<code>RemoteSyncer.Start</code> had no retry	One transient failure at cold start killed mirroring for the life of the process. The naive fix then re-registered informer handlers on retry — not an idempotent call — silently doubling event load forever	Code review, then self-review of the fix
D4	<code>deletionTimestamp</code> and <code>status.phase: Terminating</code> copied into the mirror payload	Every ordinary delete produced a burst of immutable-field rejections from the real API server	Live cluster
D5	Credential mirror scoped by namespace <i>prefix</i>	Security defect. The chart deploys with <code>--project-namespace-prefix=kubeslice</code> , so the controller's own namespace matched — its TLS key, image-pull credentials and Helm release Secret would have been mirrored	Live cluster, pre-merge
D6	A fresh Active took 31 s to advertise itself	The first publish pass ran before the Lease was acquired, so it skipped and waited a full interval. The first fix re-read the racing value and left a narrower version of the same bug	Live cluster, then its own regression test
D7	The periodic remote Lease read was unbounded	A blocked read blocks the watch loop, so <i>no</i> staleness is evaluated meanwhile. Measured at 12 s of complete silence against a stopped API server; a powered-off node would be minutes	Live cluster
D8	Promotion could be re-entered and race the renewal goroutine it had just started	Two writers to the Lease state	-race
D9	<code>--ha-mode</code> typo silently fell back to <code>standalone</code>	Unfenced dual writer. <code>--ha-mode=stanby</code> starts a second writer against the same workers — exactly the state the design exists to prevent, from one transposed letter	E2E audit
D10	Mirror stopped <i>after</i> the mode flip, as specified in both the issue and the ADR	Dual-writer window: mirrored objects carry the sync label, so the conflict guard permits overwrite and prune resurrects deletions	Design review against code

Table 10: (continued)

ID	Defect	Consequence	Found by
D11	Unbounded wait on <code>StopMirror</code>	Worse than the risk it guarded. The watch loop runs promotion inline, so “never promote into a dual writer” silently became “never promote at all”, with nothing logged after the first line	Live cluster, staged hook
D12	Promotion never published <code>status.activeController</code>	<code>PublishOnce</code> was gated on <code>IsLeader()</code> , and promotion holds the fence latch across its whole sequence. It returned nil without writing; the field kept naming the <i>dead</i> hub until the periodic loop ran	E2E with a real Active process
D13	A registered plain <code>Gauge</code> always collects, reporting 0	Role-scoped gauges published <code>ha_armed 0</code> on every Active and a 1970 timestamp on every un-promoted hub — the runbook’s own alerts would have fired forever. Fixed with <code>GaugeVecs</code> so absence is real	Live cluster (unit tests assert values, not absence)
D14	The chart’s <code>Cluster</code> CRD predates <code>status.activeController</code>	Silent pruning. The publisher logged success every 30s for four hours while nothing persisted; workers correctly saw nothing and never failed over. Anyone installing from the chart hits this	Real cluster
D15	<code>Namespace</code> status mirrored unconditionally	On a hub whose RBAC lacks <code>namespaces/status</code> , every pass fails <code>Forbidden</code> and retries forever	Real multi-cloud suite
D16	Promotion was runtime-only state	A promoted hub restarts as a Standby — and with the other also a Standby, the pair has <i>no</i> Active and both race to promote. Fixed by <code>ResumeAsActive</code> reading the local Lease at boot	Real multi-cloud suite
D17	Token Secret creation nested inside <code>if !foundSa</code>	A promoted hub error-loops forever: the <code>ServiceAccount</code> is mirrored so it is “found”, the token Secret is never created, and registration hard-errors. Proven A/B live: 28 errors in 45s before, 0 after	Code trace, proven live
D18	<code>source.Channel</code> errors on a nil channel	An unconditional <code>WatchesRawSource</code> broke every caller not wiring a kick, including the repository’s own envtest suite	Doc-comment review
D19	<code>select</code> chooses randomly among ready cases	<code>ctx.Done()</code> placed beside a send honoured cancellation about half the time	<code>-shuffle</code>
D20	<code>Dockerfile</code> never copied <code>pkg/</code> into the build context	The container image had been unbuildable since <code>pkg/ha</code> was introduced	Production-shaped deployment test

What the catalogue shows. (i) Five defects (D4, D13, D14, D15, D17) were structurally invisible to unit tests — they needed a real API server: immutable-field validation, the Prometheus collection model, CRD

pruning, real RBAC, the token controller. (ii) Three fixes contained a narrower version of the bug they fixed (D3, D6, and the first attempt at D8), consistently enough to state as a rule: *when fixing a race or a retry, check whether the fix itself re-reads the racing value or repeats a non-idempotent step.* (iii) Two defects were worse than the failure they guarded against (D11, D12) — in both, the “safe” choice converted a loud, recoverable failure into a silent permanent one; bounding every wait and failing loudly is the better default even when the bound is arbitrary. (iv) Two came from the specification itself (D10’s ordering, and #295’s resource list): specifications written before the code exists are hypotheses, and verifying them against the codebase was worth more than implementing them faithfully.

8 Known limitations and future work

Split brain is a real, observed limitation. It was declared a non-goal, and it happened. After the unplanned failover of 19 August, the old Active came back before anyone demoted it, read its *own* Lease — which it had never stopped owning — and resumed as Active. The two hubs were both Active for one hour and forty-five minutes. Nothing detects this automatically, because a hub reads only its own Lease and the two Leases are separate objects on separate API servers. The workers behaved correctly throughout: the resolver saw two hubs disagreeing and preferred the fresher `lastUpdated`, which is the anti-flap rule working as designed. The runbook documents detection and manual recovery (scale the demoted hub to zero, delete its HA Lease, scale back up). A real fix needs an arbiter — a third witness cluster or a shared consensus store — and is the single largest piece of future work.

Item	Impact	Status
Chart <code>Cluster</code> CRD lacks the HA status fields	Endpoint discovery is inert on a stock Helm install; the controller detects and reports it but cannot fix it	PR open (kubeslice #96)
Chart does not set <code>--metrics-secure=false</code>	HA metrics unscrapeable through the shipped kube-rbac-proxy on a Helm install	Not started; separate repository
Worker token delivery at registration	The Standby-valid token exists, but nothing ships it to the worker at registration time; no owner in either repository	Manual procedure documented in docs/hub-failover.md
<code>HALeadershipLost</code> and <code>HAPromotionAborted</code> never observed live	Both need a staged API server failure; unit tests only	Deterministic RBAC-based staging plan written, not executed
Asymmetric namespace deployments	The Standby watches the Active’s Lease in the same namespace name it uses itself	Would need <code>-ha-active-lease-namespace</code>
Cross-cloud data plane untested	Slice gateway pods never started on the real clusters, blocked by unrelated worker CRD skew, so tunnel-survival assertions are vacuous — and the suite says so	Out of scope; not a regression
<code>apis</code> release tag	worker-operator pins a pseudo-version of the merged commit, not a tag	Untidy, not blocking

Table 11: Open items at submission. None blocks the merged feature; the first is the only one affecting a stock installation.

Natural next steps: an arbiter for split brain; shipping the CRD fields with the chart to close the pruning class for good; automating the worker’s second credential into the registration flow; and wiring `pkg/ha` into CI — the Makefile test targets hard-code `./controllers/controller/...` and the only Go test CI step swallows failures, so the new tests are green but are not yet a gate.

9 Delivery summary

Every milestone was met on or ahead of schedule. #294 was opened a week early; the midterm demonstration on 21 July met all four criteria; and the last four weeks went into real-cluster validation, the observability and test-documentation work, and the merge.

Weeks	Planned	Delivered
1–2 (24 Jun–7 Jul)	ADR; Kind env; remote client dials	ADR PR opened 3 Jul; three-cluster Kind topology working
3–4 (8–21 Jul)	#294 fencing; #295 mirror	Both complete and live-verified. Midterm demo 21 Jul: all four criteria met
5–6 (22 Jul–4 Aug)	#297 promotion; #467	#297 delivered in four parts, all live-verified; apis repository discovered and PR opened
7–8 (5–18 Aug)	#298; #299; #468	Observability live-verified; E2E suite green; real cross-cloud validation, 92 assertions
9–10 (19–31 Aug)	Merge queue, reviews	#468/#469 delivered; chart gap found and fixed; all 23 PRs merged 29 Aug

Table 12: Planned versus delivered.

On 28 August the mentors created an `ha-integration-branch` in each repository and asked for the stack to land there rather than reviewing twenty-three pull requests individually. All were retargeted the same day; the branches were byte-identical to `master` at that point, so no diff changed. The merge order was verified conflict-free by simulating every merge locally first, which caught two traps: one PR that had to arrive through another’s merge commit rather than on its own, and one that duplicated a fix already in the chain.

Issue	Deliverable	Merged PRs
#293	Architecture decision record	controller #405
#294	Leader election + write fencing	#409
#295	Mirror engine, drift backstop, credential mirror	#411, #415, #416
#297	activeController publisher, promotion, post-promotion reconcile, token shells, audit follow-ups	#414, #417, #418, #419, #420
#298	Metrics, events, operator runbook	#425
#299	End-to-end suite, <code>make test-e2e-ha</code>	#427
#423	Test-suite documentation	#428
—	Real-cluster defect fixes; review follow-ups; Cluster conditions	#426, #429, #431
#467	Hub resolver, failover following, follow-ups	worker-op #497, #498, #502
#468	Failover robustness tests	#501
#469	Connection health conditions, events, metrics	#500
—	ActiveController and Conditions on ClusterStatus	apis #46, #47

Table 13: The twenty-three merged pull requests, by issue.

Repository	PR	Files	Lines	State
kubeslice-controller	#432	89	+15 938 / –33	Open, mergeable
worker-operator	#504	32	+3 005 / –42	Open, mergeable
apis	#48	3	+178	Open, mergeable
kubeslice (charts)	#96	1	+85	Open, mergeable

Table 14: The four final integration-to-`master` pull requests, open at submission. Each carries exactly its repository’s integration branch, including the mentors’ merge commits; nothing new was authored for them. Recommended merge order: `apis`, then `worker-operator` (its `go.mod` pins the `apis` commit), then the controller, then the charts.

10 Conclusion

The KubeSlice Controller can now survive the loss of its entire hub cluster. A second hub tracks the first continuously, detects its failure across the cluster boundary within a configurable budget, promotes itself under three independent guards, and publishes its own endpoint so that workers reconnect on their own. Measured failover on real infrastructure in two different clouds is 25 seconds, with workers writing to the promoted hub within about a minute. Deployments that do not opt in behave exactly as they did before. The work spans four repositories and twenty-three merged pull requests, backed by 227 tests — 88.4% statement coverage in the core `pkg/ha` package — an end-to-end suite, and 289 assertions across real multi-cluster runs.

The most useful output may not be the feature. It is the defect catalogue: twenty real bugs, five structurally invisible to unit testing, three introduced by the fix for an earlier one, and two where the cautious choice

silently produced a worse outcome than the risk it was avoiding. Several came from specifications that were wrong because they were written before the code was read.

And the evidence that mattered most was not arranged. On a night in August, a cloud provider reclaimed the Active hub's nodes without warning. Twenty-one seconds later the Standby had promoted itself, confirmed the failure was genuine rather than a network partition, and taken over. Both workers followed. Nobody was watching.

A Running the tests and deploying a hub pair

```
# --- Controller -----
go test -race -count=1 -cover ./pkg/ha/...      # unit tests, 88.4% coverage
go test -race -count=2 -shuffle=on ./pkg/ha/...  # catches ordering dependence
make test-e2e-ha                                # creates + destroys 2 Kind clusters
# --- Worker operator -----
go test -race -count=1 -cover ./pkg/hub/resolver/... ./pkg/hub/failover/...
```

`make test` and `make unit-test` are not usable in these repositories today: the Makefile targets hard-code `./controllers/controller/...`, `go vet ./service/...` fails on `master`, and the `envtest` suites need control-plane binaries fetched with `make envtest`. These are pre-existing conditions, which is why the commands above name packages explicitly.

```
# 1. On the STANDBY cluster, store a kubeconfig for the ACTIVE hub
kubectl --context active config view --minify --flatten > active.kubeconfig
kubectl --context standby -n kubeslice-controller create secret generic \
  ha-active-kubeconfig --from-file=kubeconfig=active.kubeconfig

# 2. ACTIVE hub Deployment args
--ha-mode=active --ha-identity=hub-active-1
--controller-end-point=https://<active-api-server>:6443

# 3. STANDBY hub Deployment args (mount the Secret at this path)
--ha-mode=standby --ha-identity=hub-standby-1
--ha-active-kubeconfig=/etc/kubeslice/ha/kubeconfig
--controller-end-point=https://<standby-api-server>:6443

# 4. Grant the Standby's remote identity read access ON THE ACTIVE HUB,
# then bind it -- the shipped ClusterRoleBinding subject is a placeholder
kubectl --context active apply -f config/ha/active-cluster-clusterrole.yaml

# 5. Confirm
kubectl --context active -n kubeslice-controller get lease kubeslice-controller-ha
kubectl --context standby logs deploy/kubeslice-controller-manager | grep "watching active hub"
kubectl --context active get cluster <name> -o jsonpath='{.status.activeController}'
```

Before relying on this on a Helm-installed cluster, confirm the installed `Cluster` CRD actually carries `status.activeController`. If it does not, the API server prunes every write silently and workers will never fail over. The controller reports this through `ha_active_publish_errors_total` and a log line naming the likely cause; the permanent fix is [kubeslice #96](#).